

Field-level inference: Bayesian hierarchical model for joint field-parameter sampling

Sacha Guerrini

2025-08-21

```
import numpy as np
import os

os.environ["JAX_PLATFORM_NAME"] = "cpu" #"cpu" if you don't have access to a GPU
import urllib.request
import jax
import jax.numpy as jnp
from jax.scipy.stats import norm as normal
import scipy.linalg as la
from cycler import cycler
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from matplotlib.colors import LogNorm, SymLogNorm
import matplotlib.patches as mpatches
import matplotlib.lines as mlines
from mpl_toolkits.axes_grid1 import make_axes_locatable

print(f"Device used by Jax: {jax.devices()[0]}")

np.random.seed(123456)
```

Device used by Jax: TFRT_CPU_0

Note on the version

This document is generated using [jax](#) and its sampling library [blackjax](#). [jax](#) is a rapidly evolving framework. To reproduce this document, one should use the following versions of the software:

- `jax`: 0.5.0
- `blackjax`: 1.2.5
- `jaxili`: <https://github.com/sachaguer/jaxili/tree/develop>

Context

In this exercise, we will illustrate field-level inference using the BBKS cosmological model. We will focus on three parameters:

- A_s : the amplitude of the primordial power spectrum
- n_s : the spectral index of the primordial power spectrum
- f_{NL} : a scalar parameter characterising the primordial non-Gaussianities

The primordial gravitational potential Φ_L is a Gaussian random field with zero mean and power spectrum:

$$P(k) = A_s k^{n_s-1}.$$

This primordial potential is mapped to a ‘non-linear’ potential field:

$$\Phi_{\text{NL}} = \Phi_L + f_{\text{NL}} \Phi_L^2.$$

The non-linear potential and the density contrast field are linked by a transfer function in Fourier space,

$$\Phi_{\text{NL}}(k) = \mathcal{F}[\Phi_{\text{NL}}(x)], \quad \delta(k) = D_1 \sqrt{k} T(k) \phi_{\text{NL}}(k),$$

where $\mathcal{F}$ is the Fast Fourier Transform (FFT) operator, $T(k)$ is the transfer function, and D_1 is the linear growth factor (in arbitrary units).

The transfer function is modeled using:

$$T(k) = \frac{\log(1 + \alpha q)}{\alpha q} (1 + \beta q + (\gamma q)^2 + (\delta q)^3 + (\epsilon q)^4)^{-1/4}, \quad q \equiv \frac{k}{\Gamma},$$

with shape parameter $\Gamma = \Omega_m h \exp(-\Omega_b - \sqrt{2h} \Omega_b / \Omega_m)$. Other cosmological parameters are fixed to the following values:

Problem parameters

```
N = 32
L = 1.0 # box size
# The following parameters will be our fiducial values
A_s = 6e-9 # power spectrum normalisation, arbitrary units
n_s = 0.96 # spectral index
f_NL = 2000.0 # non-linear coupling parameter
D1 = 1.732e7 # growth factor of fluctuations at z=0, arbitrary units
```

Cosmological parameters

```
Omega_b = 0.049
Omega_m = 0.315
h = 0.674
fb = Omega_b / Omega_m
```

BBKS parameters

```
shape = Omega_m * h * np.exp(-Omega_b - np.sqrt(2.0 * h) * fb)
alpha = 2.34
beta = 3.89
gamma = 16.1
delta = 5.46
epsilon = 6.71
```

When $f_{\text{NL}} = 0$, the model is linear; therefore, $\delta(x)$ is a Gaussian random field with the BBKS power spectrum:

$$P_\delta(k) = D_1^2 k T^2(k) P(k) = A_s D_1^2 k^{n_s} T^2(k).$$

We model the data d as a noisy observation of the δ field. We assume that the noise is zero-mean, additive and Gaussian, i.e.

$$d(x) = \delta(x) + n(x), \quad n(x) \sim \mathcal{N}(0, N),$$

where N is the noise covariance matrix.

The figures below illustrate the model and the data generation process. It has been generated for fiducial parameters and will be used as observed data in what follows.

```
<>:9: SyntaxWarning: invalid escape sequence '\m'
<>:16: SyntaxWarning: invalid escape sequence '\m'
<>:9: SyntaxWarning: invalid escape sequence '\m'
<>:16: SyntaxWarning: invalid escape sequence '\m'
/tmp/ipykernel_492461/926910477.py:9: SyntaxWarning: invalid escape sequence '\m'
    ax0.set_title("$\\Phi_{\\mathrm{L}}$")
/tmp/ipykernel_492461/926910477.py:16: SyntaxWarning: invalid escape sequence '\m'
    ax1.set_title("$\\Phi_{\\mathrm{NL}}=\\Phi_{\\mathrm{L}}+f_{\\mathrm{NL}}\\Phi_{\\mathrm{L}}^2$")
```

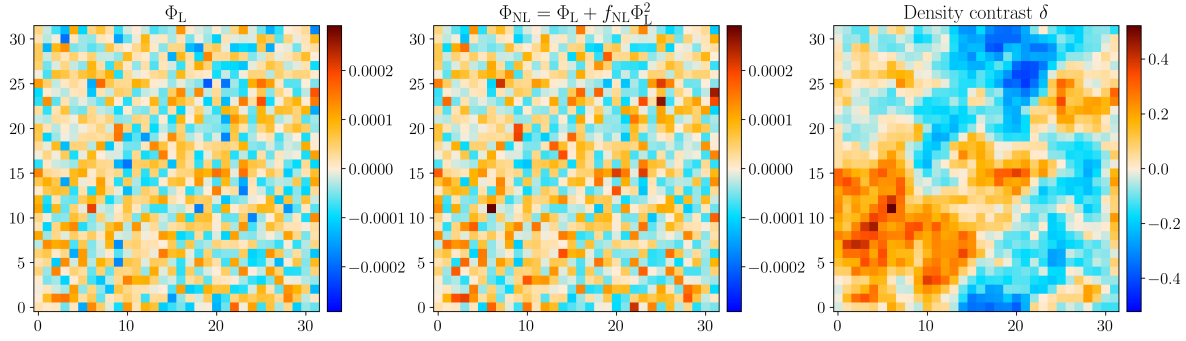


Figure 1: Visualisation of the primordial potential and the density contrast field

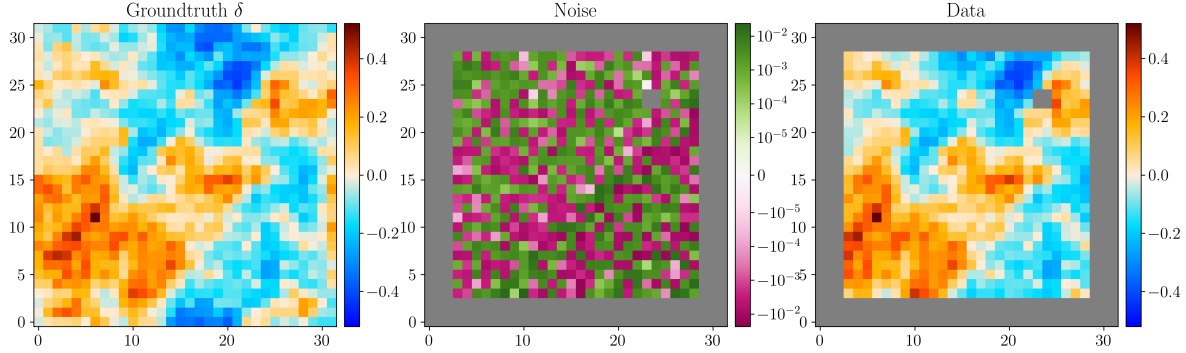


Figure 2: Visualisation of the observation. *Left*: the groundtruth density contrast, *Middle*: the mask and noise matrix, *Right*: the observed data.

Inferring the parameters using the field

Using Bayesian inference, we want to estimate the parameters $\theta = (A_s, n_s, f_{NL})$ given the data d and the noise variance field N .

To do so, we use Bayes' theorem:

$$p(\theta|d) \propto p(d|\theta)p(\theta).$$

Here our data d is the observed field, hence the name field-level inference. In the previous section, we have build a Bayesian Hierarchical Model (BHM) for the data, which specifies the likelihood.

However, the likelihood is defined at the pixel level. We have access to $p(\theta, z|d)$ where z are the primordial pixel values. In practice, to get constraints on the cosmological parameters only, we must marginalise on pixel values z .

$$p(\theta|d) = \int p(\theta, z|d) dz.$$

This integral is intractable in practice. Sampling cosmological parameters and initial conditions is a challenging task as the problem is very high dimensional. It requires sampling techniques that are not sensitive to the curse of dimensionality such as Hamiltonian Monte Carlo.

A first task is to write the log-prior, log-likelihood and log-posterior functions for the BHM.

We can summarise the model using the following diagram:

In what follows, we will use the following values for the prior of the cosmological parameters. The likelihood is explicitly defined via the choices of distribution from which the parameters (prior) and latent variables are sampled from. Probabilistic programming languages such as `pyro` or `numpyro` implement BHMs following the graph structure sketched in Figure 3.

```
# Define hyperparameters for the prior
A_s_scale = 6e-9
f_NL_scale = 2e3
mu_A_s = 0.9 * A_s_scale
sigma_A_s = 0.1 * A_s_scale
mu_n_s = 0.95
sigma_n_s = 0.01
mu_f_NL = 0.75 * f_NL_scale
sigma_f_NL = 0.25 * f_NL_scale
```

The output of the log-prior, log-likelihood and log-posterior functions is a scalar value and can be found for various examples below.

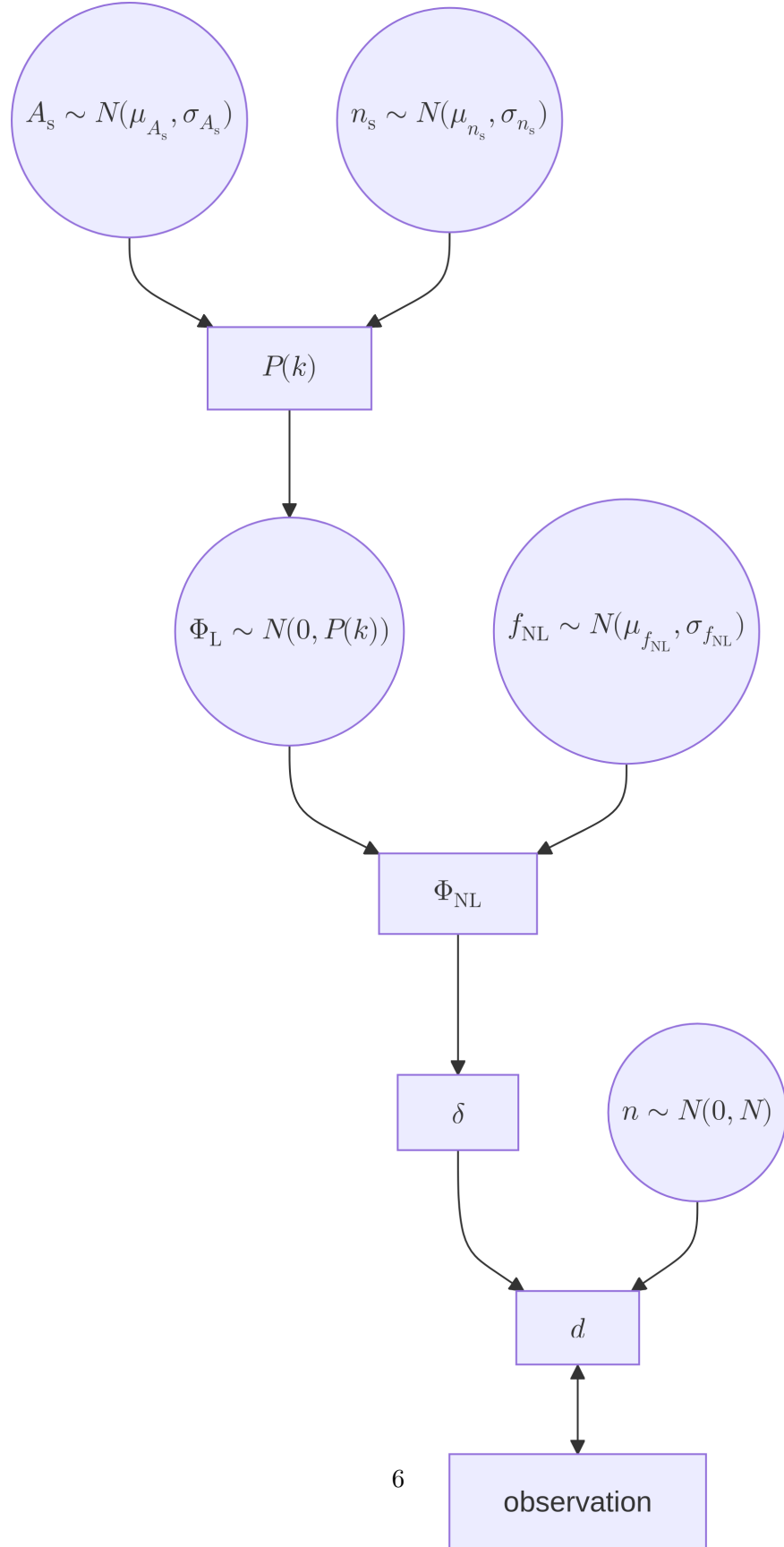


Figure 3: Bayesian hierarchical model for field-level inference. Round boxes are sampled from and rectangular boxes are derived.

```
# Test the functions with the ground truth white noise field
theta = pack_theta(A_s=A_s, n_s=n_s, f_NL=f_NL, field=white_noise)
log_prior(theta), log_likelihood(theta, data, noise_variance_field), log_posterior(
    theta, data, noise_variance_field
)
```

```
(Array(-458.7011, dtype=float32),
 Array(-554.5359, dtype=float32),
 Array(-1013.237, dtype=float32))
```

```
# Generate a new white noise field and compute the log-likelihood
white_noise_2 = jax.random.normal(jax.random.PRNGKey(2), (N, N))
theta_2 = pack_theta(A_s=A_s, n_s=n_s, f_NL=f_NL, field=white_noise_2)
log_likelihood(theta_2, data, noise_variance_field), log_prior(theta_2), log_posterior(
    theta_2, data, noise_variance_field
)
```

```
(Array(-2063638.9, dtype=float32),
 Array(-536.76605, dtype=float32),
 Array(-2064175.6, dtype=float32))
```

```
# Change the cosmological parameters and compute the log-likelihood
A_s_new = 7e-9
n_s_new = 0.98
f_NL_new = 2500.0
theta_3 = pack_theta(A_s=A_s_new, n_s=n_s_new, f_NL=f_NL_new, field=white_noise_2)
log_likelihood(theta_3, data, noise_variance_field), log_prior(theta_3), log_posterior(
    theta_3, data, noise_variance_field
)
```

```
(Array(-2448481.8, dtype=float32),
 Array(-545.3216, dtype=float32),
 Array(-2449027., dtype=float32))
```

Sample the posterior using field-level inference (explicit)

To efficiently sample parameters in a high-dimensional space, one must rely on gradient-based Markov Chain Monte Carlo (MCMC) techniques such as HMC. `jax` is a powerful library that uses autodifferentiation to compute gradient of functions in such a way that, given the log-likelihood function is written in `jax`, we have access to it for free.

In what follows, we use the No-U-Turn Sampler (NUTS) from the `blackjax` library, which is a JAX implementation of the NUTS algorithm to sample the cosmological parameters and the initial conditions of the field at the same time. `jax` can run automatically on GPU. In this document, we ran on CPU on a laptop and sampling five chains of 10,000 samples each takes around two hours. I have not checked the speed up obtained by running on GPU.

It is possible to speed up the sampling by using Gibbs sampling to sample the field with HMC and the cosmological parameters with e.g. slice sampling. The implementation of it currently does not perform correctly. In what follows, diagnostics of the chains will be presented using the NUTS algorithm.

Why is it more efficient that way?

Diagnose the chains

Log-likelihood

```
<>:13: SyntaxWarning: invalid escape sequence '\m'
<>:13: SyntaxWarning: invalid escape sequence '\m'
/tmp/ipykernel_492461/3802684793.py:13: SyntaxWarning: invalid escape sequence '\m'
    ax.set_ylabel("$-\log \mathcal{L}$")
```

Trace plots

We can then check the trace plots for parameters such pixels and cosmological parameters.

Sequential posterior power spectrum

Given our samples, we can do posterior predictive checks to further ensure that the chain has converged to the correct posterior. To do so, one needs to run the data model using the samples as input. We perform posterior predictive checks by looking at the power spectrum of the reconstructed signal and the ground truth. In practice, the groundtruth signal will not be available but we can compare against perturbation theory predictions.

Effective sample size in the chains

When running an MCMC, samples are not independent. The effective sample size (ESS) is a measure of how many independent samples we have in our MCMC chain. It can be computed using the integrated autocorrelation time, which quantifies the correlation between samples at different lags. We here compute the ESS using function available in the [emcee documentation](#). As the error of an MCMC estimator reduces with the number of samples, it is important to have a sufficiently large ESS.

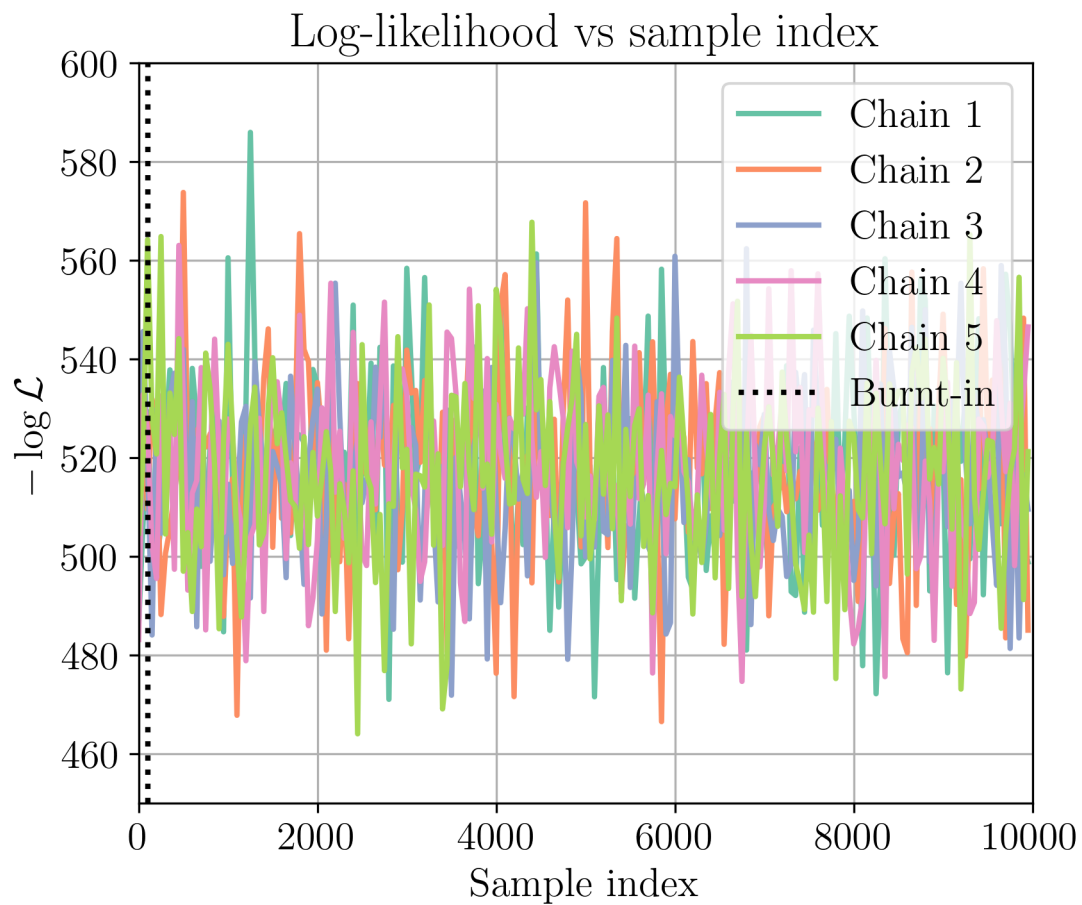


Figure 4: Log-likelihood trace plot for each chain. The dashed line indicates a burn-in period of a hundred samples.

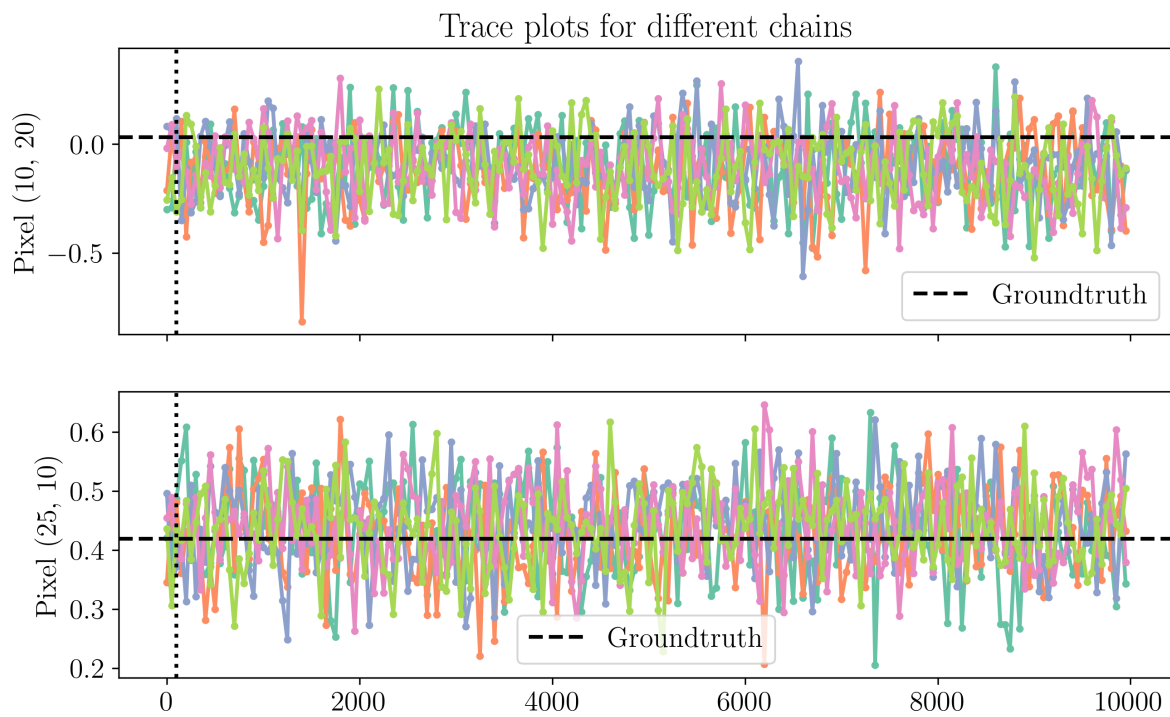


Figure 5: Trace plots for the pixel values at (10, 20) and (25, 10) for each chain. The dashed line indicates the groundtruth value.

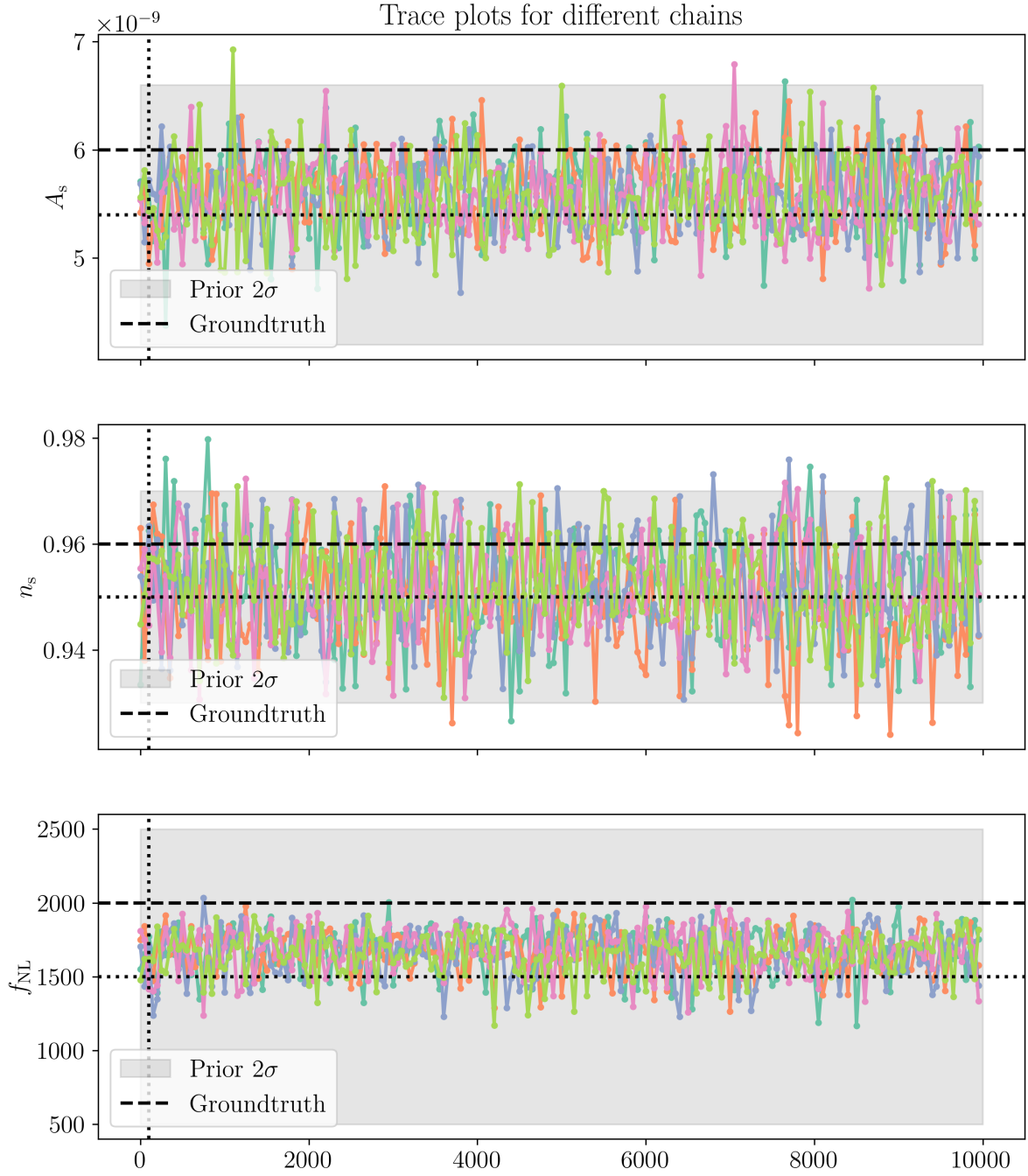


Figure 6: Trace plots for the cosmological parameters A_s , n_s , and f_{NL} for each chain. The dashed line indicates the groundtruth value and the shaded area the 2σ region of the Gaussian prior.

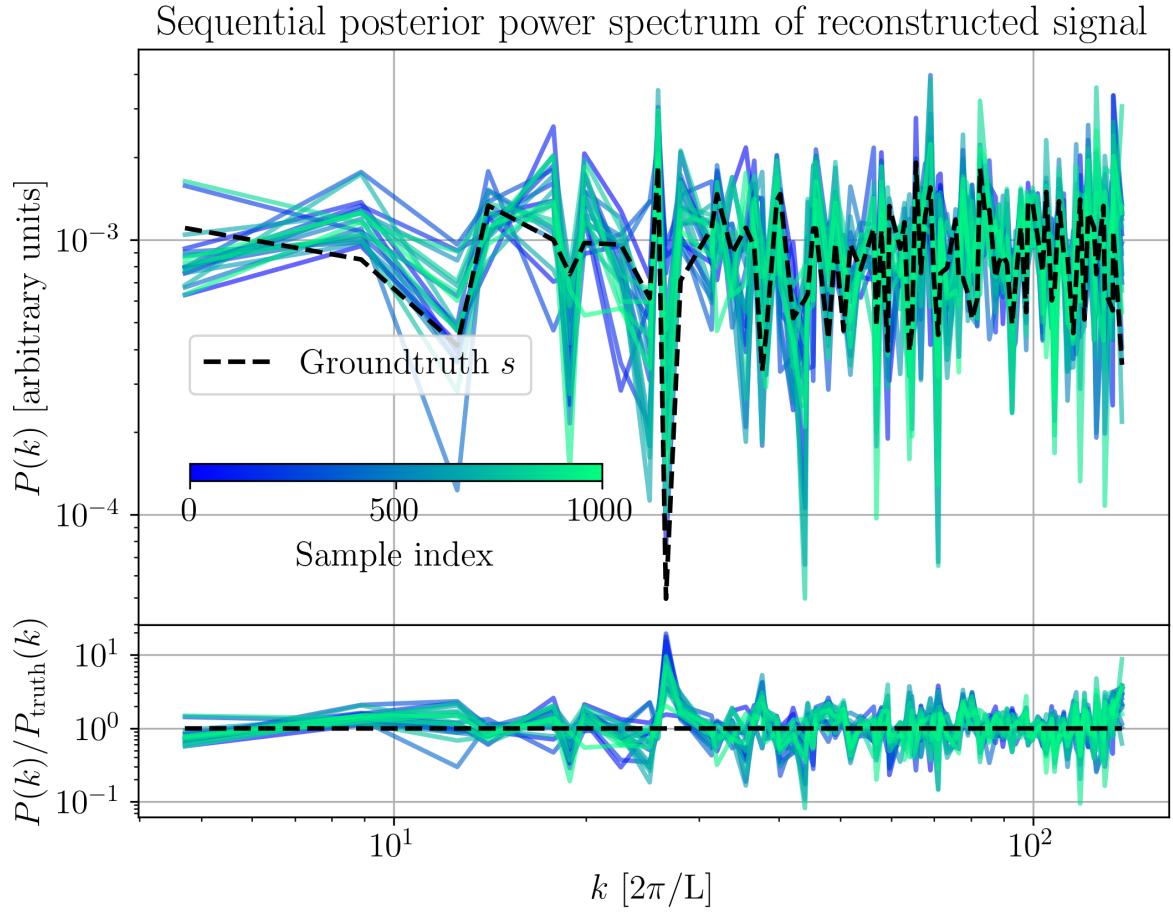


Figure 7: Sequential posterior power spectrum of the reconstructed signal. The dashed line indicates the groundtruth power spectrum.

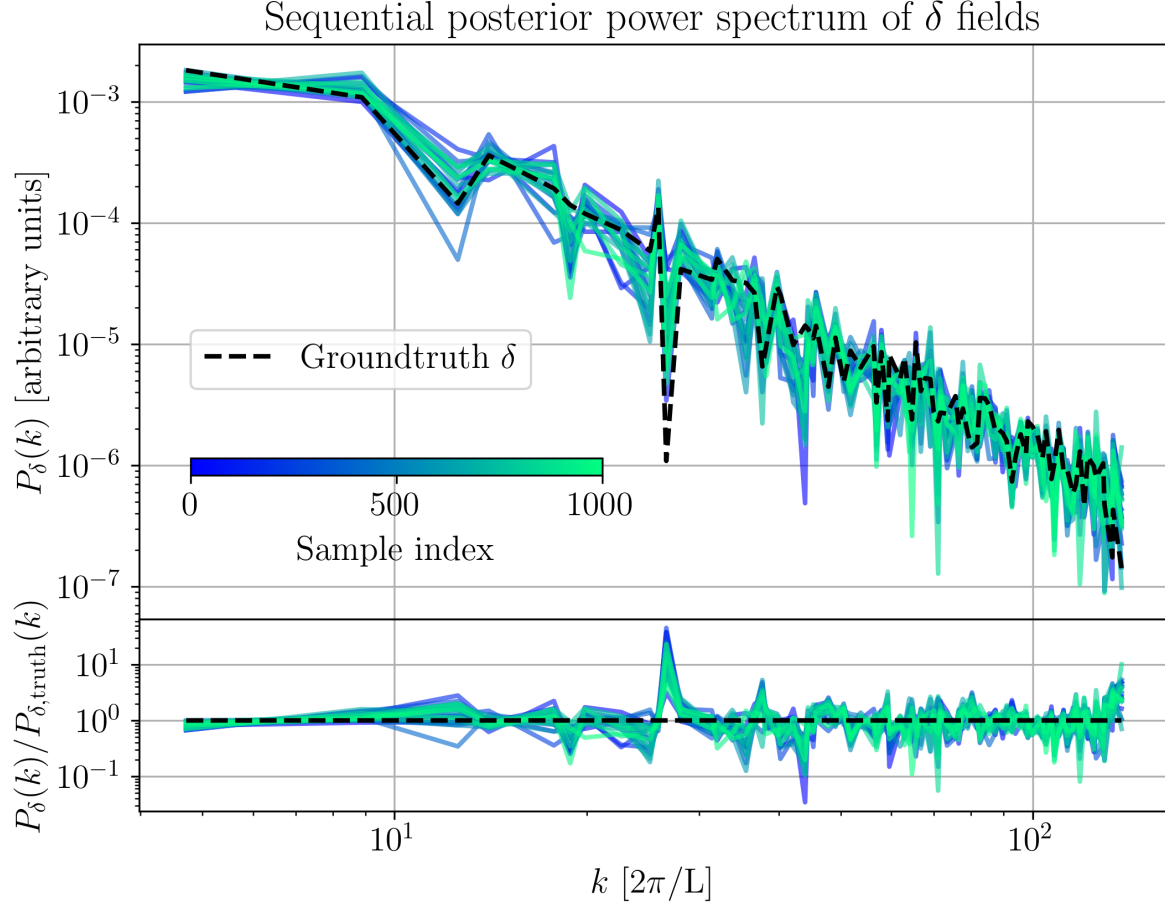


Figure 8: Sequential posterior power spectrum of the reconstructed δ field. The dashed line indicates the groundtruth power spectrum of the δ field.

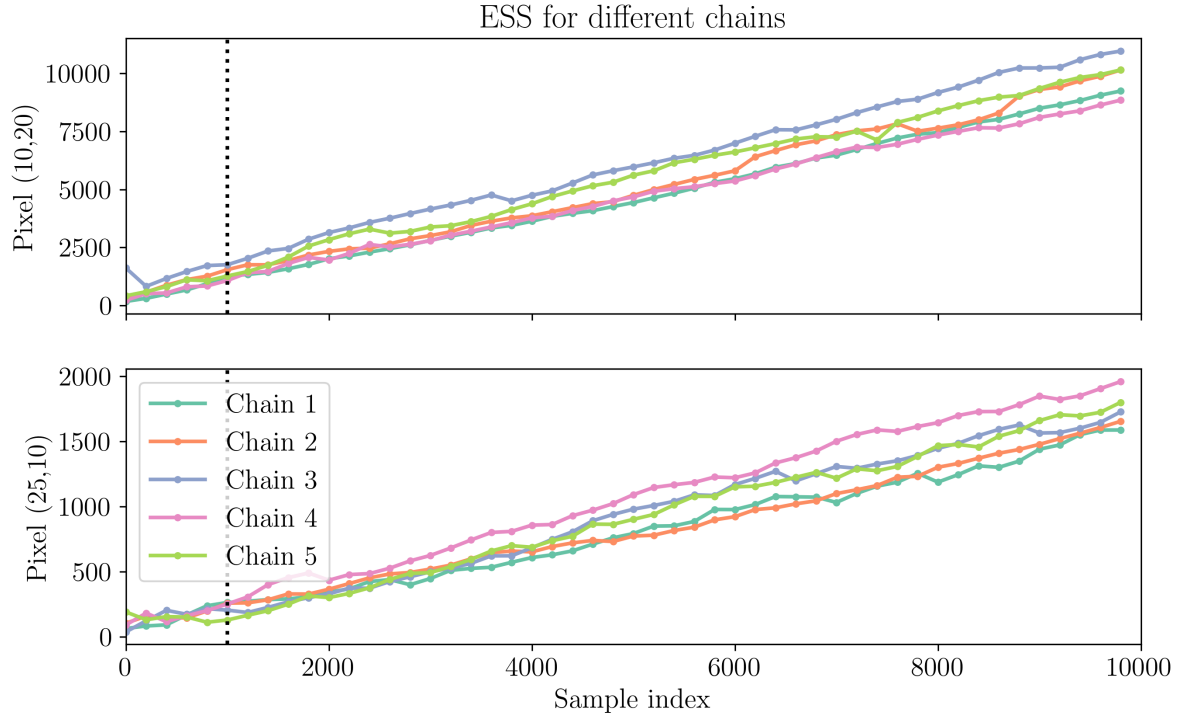


Figure 9: Effective sample size (ESS) for the pixels (10, 20) and (25, 10) in the field for each chain. The dashed line indicates the burn-in period.

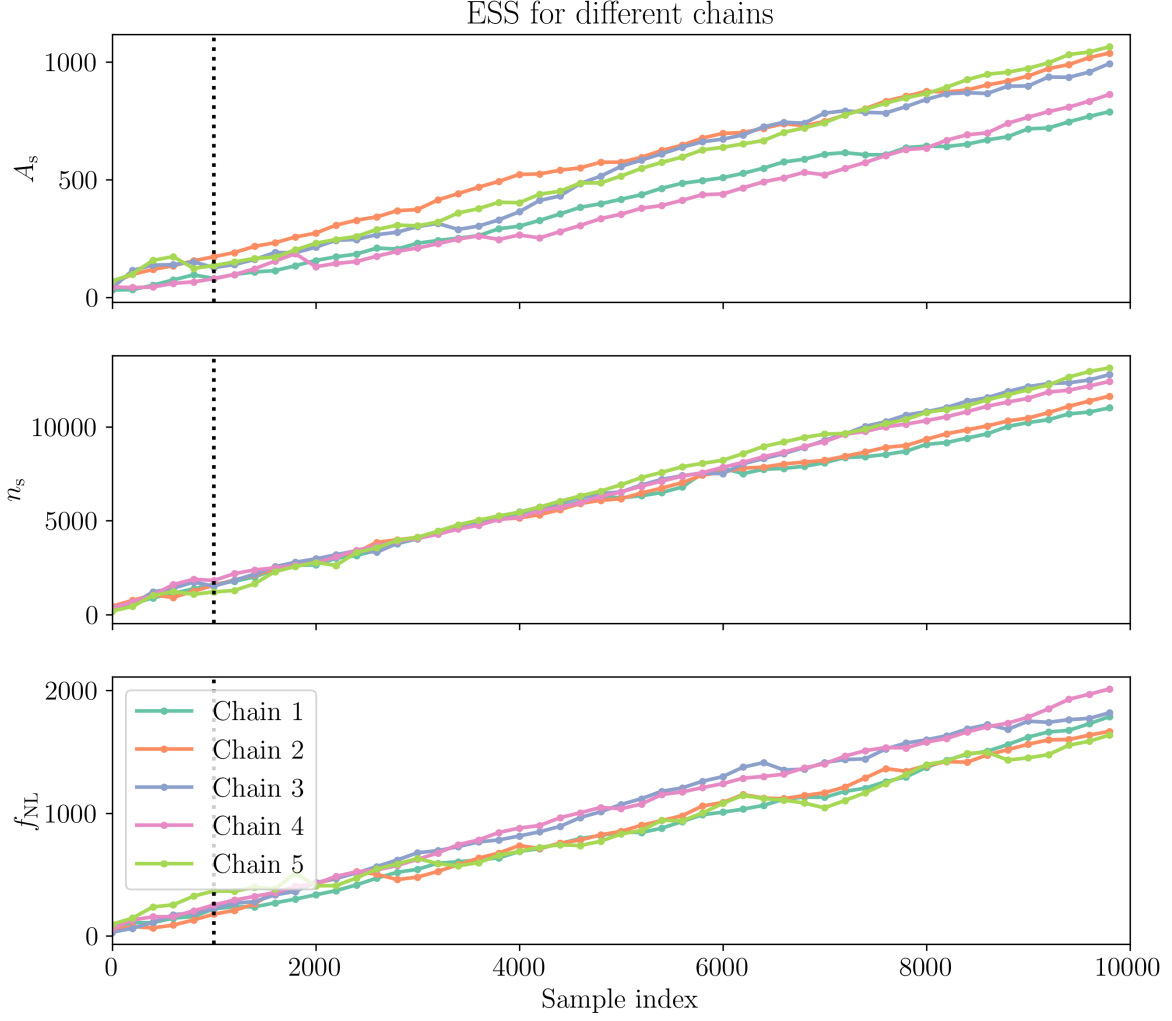


Figure 10: Effective sample size (ESS) for the cosmological parameters A_s , n_s , and f_{NL} for each chain. The dashed line indicates the burn-in period.

Gelman-Rubin test

Another useful test is the Gelman-Rubin (GR) test. It assesses if the samples are uncorrelated by computing the ratio between the variance within the chains and between multiple independent chains.

Parameters

- m : number of chains
- n : number of samples per chain

Definitions

- “between” chains variance:

$$B \equiv \frac{n}{m-1} \sum_{j=1}^m (\bar{\psi}_{\cdot j} - \bar{\psi}_{\cdot\cdot})^2 \quad \text{where} \quad \bar{\psi}_{\cdot j} = \frac{1}{n} \sum_{i=1}^n \psi_{ij} \quad \text{and} \quad \bar{\psi}_{\cdot\cdot} = \frac{1}{m} \sum_{j=1}^m \bar{\psi}_{\cdot j} \quad (1)$$

- “within” chains variance:

$$W \equiv \frac{1}{m} \sum_{j=1}^m s_j^2 \quad \text{where} \quad s_j^2 = \frac{1}{n-1} \sum_{i=1}^n (\psi_{ij} - \bar{\psi}_{\cdot j})^2 \quad (2)$$

Estimators:

Estimators of the marginal posterior variance of the estimand:

- $\widehat{\text{var}}^- \equiv W$: underestimates the variance
- $\widehat{\text{var}}^+ \equiv \frac{n-1}{n}W + \frac{1}{n}B$: overestimates the variance

Test:

- Potential scale reduction factor: $\widehat{R} \equiv \sqrt{\frac{\widehat{\text{var}}^+}{\widehat{\text{var}}^-}}$
- Test: $\widehat{R} \rightarrow 1$ as $n \rightarrow \infty$

```
# The Gelman-Rubin function expects (n_chains, n_samples, n_variates)
Rhat = gelman_rubin(jnp.array([samples_chain[c] for c in range(N_chains)]))

# Run Gelman-Rubin
Rhat_A_s = Rhat[0]
Rhat_n_s = Rhat[1]
Rhat_fNL = Rhat[2]
Rhat_field_flat = Rhat[3:] # shape: (N*N,)
Rhat_field = Rhat_field_flat.reshape((N, N))

print("Gelman-Rubin stats:")
```



```
print("A_s:", Rhat_A_s)
print("n_s:", Rhat_n_s)
print("fNL:", Rhat_fNL)
```

Gelman-Rubin stats:

A_s: 1.0003575

n_s: 0.9999679

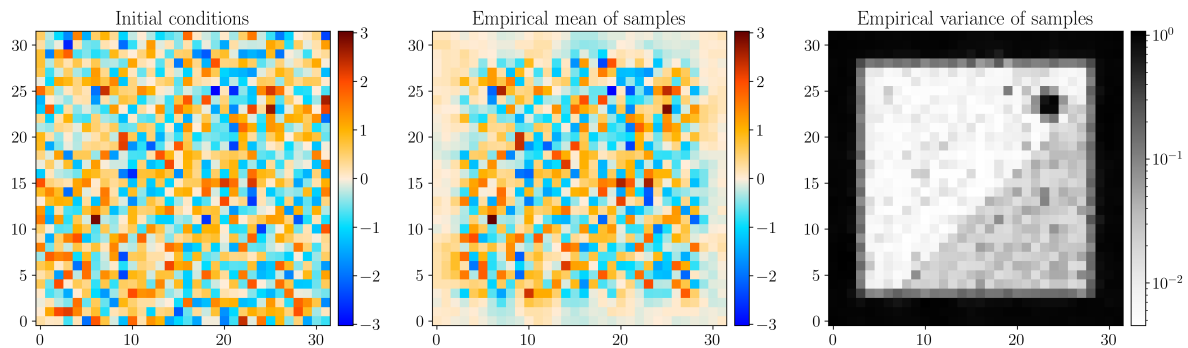
fNL: 1.0003572

```
<>:4: SyntaxWarning: invalid escape sequence '\h'
<>:4: SyntaxWarning: invalid escape sequence '\h'
/tmp/ipykernel_492461/2195658240.py:4: SyntaxWarning: invalid escape sequence '\h'
    ax.set_title("Gelman-Rubin statistic $\hat{R}$")
```

Visualise summaries of the chains

Now that we have carefully checked the convergence of the MCMC chains, we can visualise the results.

First, we can use the samples to reconstruct the δ field and check that it visually matches the truth.



```
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:17: SyntaxWarning: invalid escape sequence '\m'
<>:29: SyntaxWarning: invalid escape sequence '\m'
<>:10: SyntaxWarning: invalid escape sequence '\m'
<>:17: SyntaxWarning: invalid escape sequence '\m'
<>:29: SyntaxWarning: invalid escape sequence '\m'
/tmp/ipykernel_492461/3114878575.py:10: SyntaxWarning: invalid escape sequence '\m'
    ax0a.set_title("Groundtruth $\Phi_{\mathrm{L}}$")
```

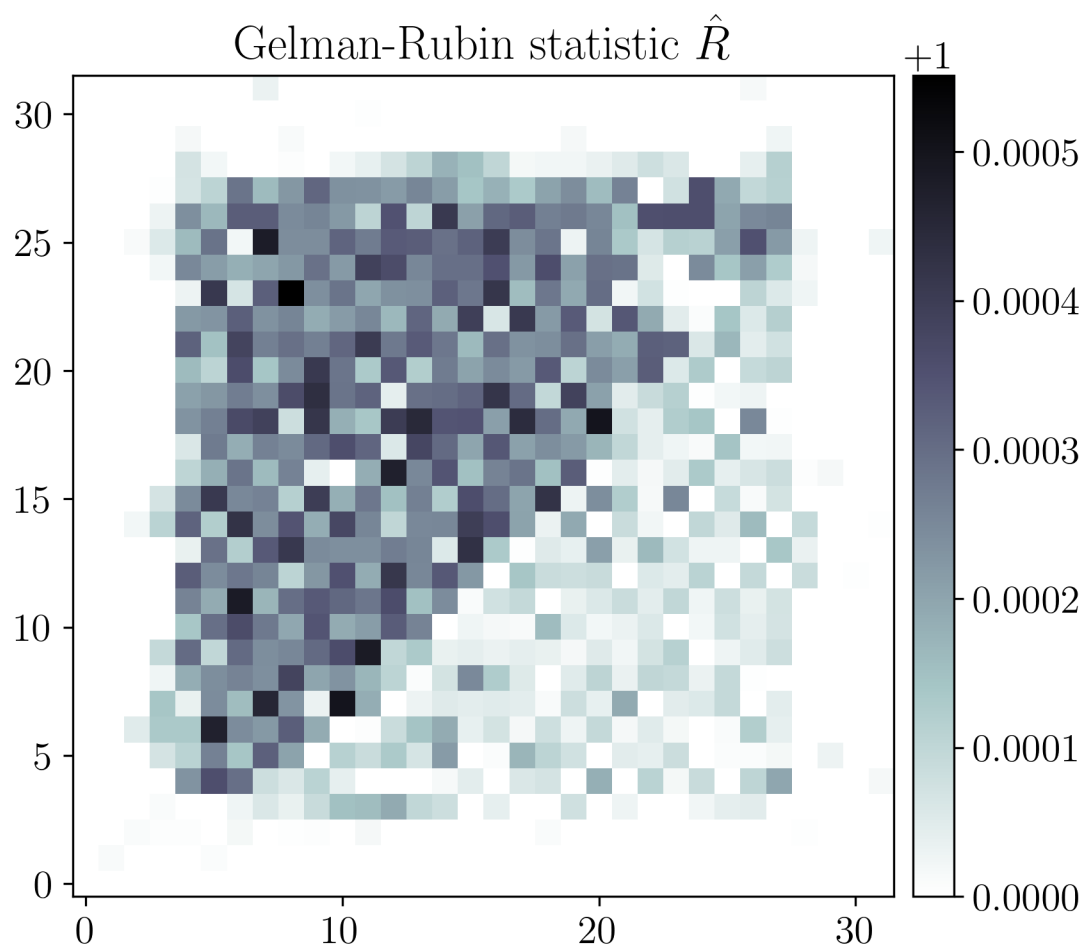


Figure 11: Gelman-Rubin statistic $\hat{R}$ for the field.

```
/tmp/ipykernel_492461/3114878575.py:17: SyntaxWarning: invalid escape sequence '\m'  
    ax1a.set_title("Empirical mean of  $\Phi_{\mathrm{L}}$  samples")  
/tmp/ipykernel_492461/3114878575.py:29: SyntaxWarning: invalid escape sequence '\m'  
    ax2a.set_title("Empirical variance of  $\Phi_{\mathrm{L}}$  samples")
```

We can finally visualise the posterior samples in the cosmological parameters space.

This result has been obtained sampling all latent variables. It is also possible to extract information at the field-level without relying on high-dimensional sampling of the initial conditions.

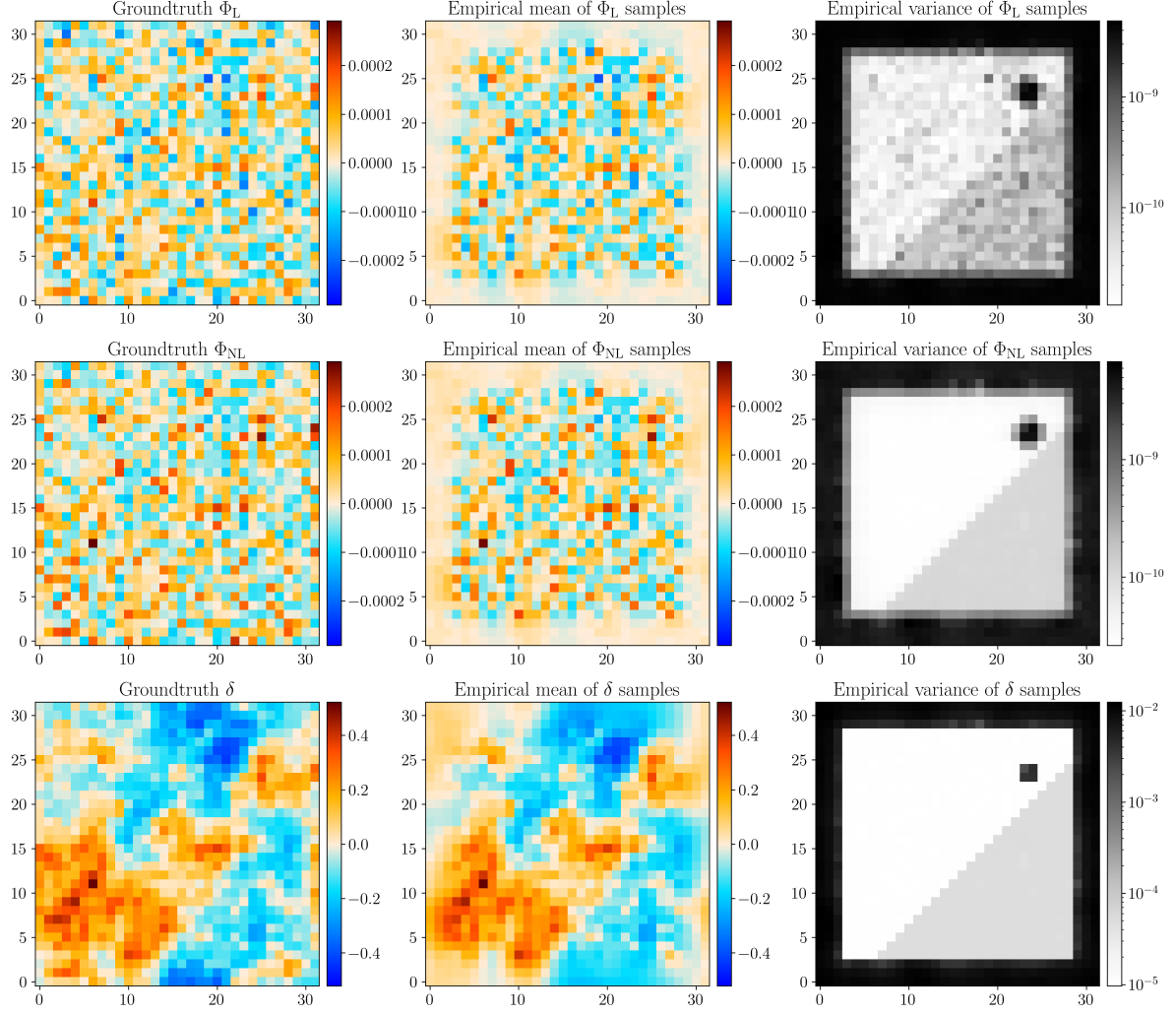


Figure 12: Reconstruction of the Φ_L , Φ_{NL} , and δ fields from the initial conditions and cosmological parameters samples. The first row shows the initial conditions, the second row the groundtruth fields, the third row shows the empirical mean of the samples, and the fourth row shows the empirical variance of the samples.

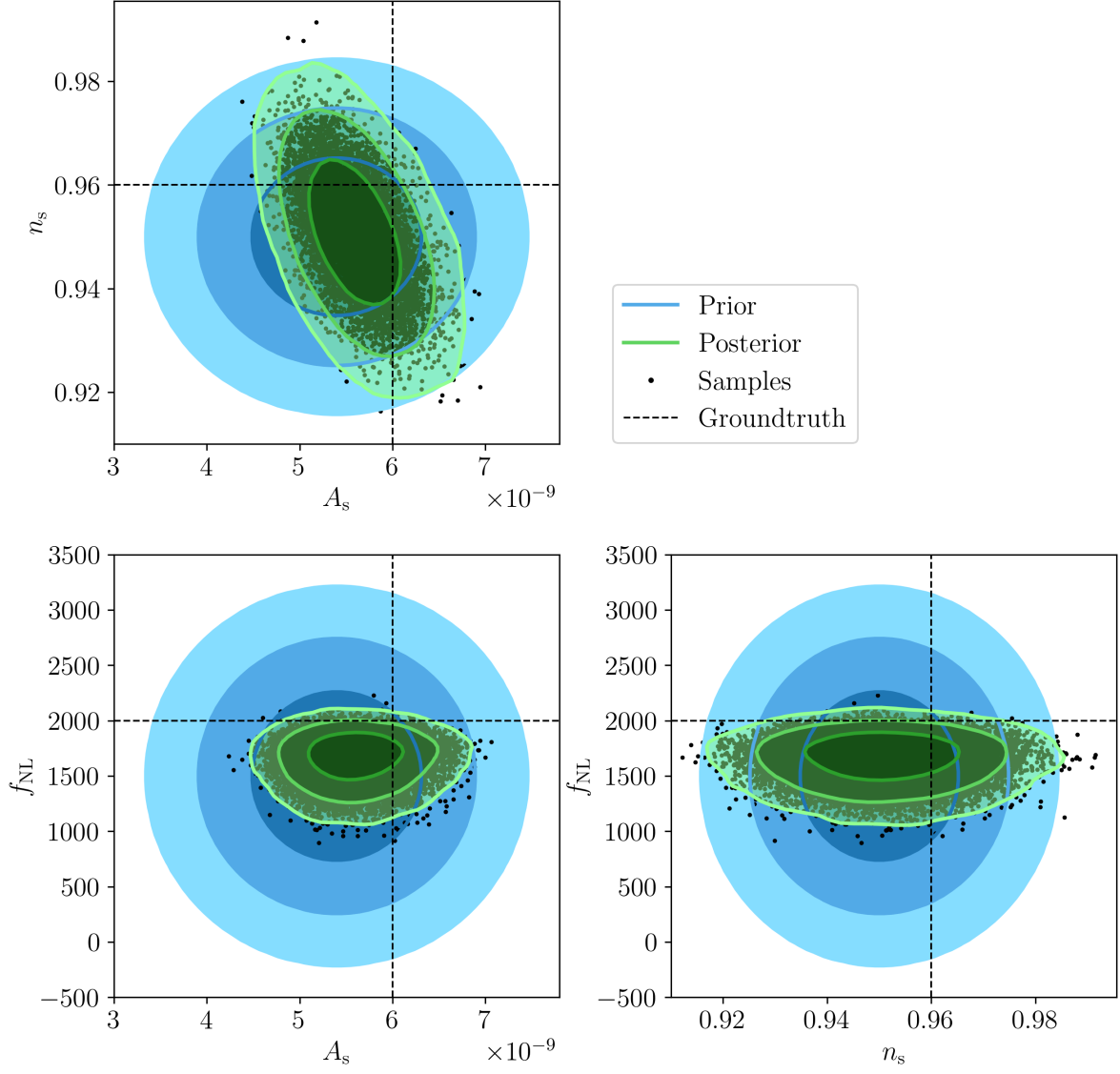


Figure 13: Posterior samples in the cosmological parameters space. The blue contours show the prior distribution, while the green filled contours show the posterior distribution. The lines indicate the one, two, and three σ confidence level. The dashed lines indicate the groundtruth.